

Milestone 1 — Requirements and Results

CSE480 Machine Vision · Ain Shams University · Mechatronics Engineering

This report maps every requirement of the Milestone 1 brief to the code that fulfils it and to the measured result. Every requirement of Milestone 1 is now met; what is **not** achieved is a complete reconstruction, and §10 shows by measurement why. Milestone 2 remains substantially outstanding (§12). Every figure quoted is reproducible from the commands in §13.

A typeset version is `report/milestone_1_report.pdf`; a web version with the figures is at <https://claude.ai/code/artifact/a42f5dbc-e48e-4d42-9d0f-b4c8bb7a7746>.

1. Summary

Automated conformance checks passing	24 of 24
Unit tests passing	108
Puzzle	5×7, 35 pieces, 58 internal seams
Blind reconstruction — position accuracy	24 / 35 cells (68.6%)
Blind reconstruction — orientation accuracy	23 / 35 cells (65.7%)
Blind reconstruction — neighbour accuracy	34 / 58 seams (58.6%)
Complete reconstruction	No
End-to-end runtime	≈32 s

The classical library is complete and every operation the brief requires from scratch is implemented, tested and exercised on the real photographs. The end-to-end routine accepts a scrambled puzzle with unknown positions and orientations, infers the grid shape itself, fills every cell, and returns a reconstructed image together with a numerical quality score.

It does **not** achieve a complete reconstruction. A third of the pieces land in the wrong cell. §10 shows, by measurement, that this is a limitation of the compatibility measure rather than of the assembly search, and §11 records how the last two conformance checks were closed.

2. How conformance is measured

`scripts/audit_requirements.py` is not a checklist of names. Each check imports the module, calls the function on real or synthetic data, and validates the result — the convolution check compares against an explicit double loop, the boundary tracer is compared point-

for-point against `cv2.findContours`, and the assembly check cuts a known image into a real interlocking puzzle, scrambles it, and requires exact reconstruction.

```
python3 -m scripts.audit_requirements      # 24/24
python3 -m pytest -q                       # 108 tests, ~15 s
```

3. Milestone 1 §1 — Image Enhancement

"The library must provide, from scratch: noise reduction by mean, Gaussian (kernel derived from size and standard deviation) and median filtering, any loop required by the median filter justified; contrast adjustment by histogram equalisation and by contrast stretching, with the underlying histogram computed by the library; sharpening by unsharp masking or a Laplacian operator built on the convolution routine; thresholding by global, Otsu and adaptive (mean or Gaussian) methods, used subsequently to separate pieces from the background."

Requirement	Implementation	Evidence	Status
Convolution from scratch	<code>enhancement.convolution</code>	Verified against an explicit double loop; rejects even-sized kernels	Met
Mean filter	<code>enhancement.mean_filter</code>	Built on <code>convolution</code>	Met
Gaussian from size + σ	<code>enhancement.gaussian_kernel</code> , <code>gaussian_filter</code>	Kernel derived from size and σ , normalised, verified symmetric	Met
Median filter, loop justified	<code>enhancement.median_filter</code>	Impulse noise removed; the loop is justified in the docstring — a median is not a convolution and cannot be expressed as one	Met
Histogram computed by the library	<code>enhancement.histogram</code>	Used by both contrast operations	Met
Histogram equalisation	<code>enhancement.histogram_equalization</code>	Output verified to span the full range	Met
Contrast stretching	<code>enhancement.contrast_stretch</code>		Met
Sharpening on the convolution routine	<code>enhancement.unsharp_mask</code> , <code>laplacian_kernel</code> , <code>laplacian_sharpen</code>	Both an unsharp mask and a Laplacian, both built on <code>convolution</code>	Met
Global threshold	<code>thresholding.global_threshold</code>	With inversion	Met
Otsu	<code>thresholding.otsu_threshold</code>	Returns both the mask and the chosen threshold	Met
Adaptive, mean and Gaussian	<code>thresholding.adaptive_threshold</code>	Both variants	Met
Used to separate pieces from the background	<code>segmentation.foreground_mask</code> , <code>frame_extraction</code>	On a real photograph: 26.7% foreground at Otsu threshold 93	Met

OpenCV is used only for image I/O, colour conversion and geometric warps. Every operation listed above is plain NumPy.

4. Milestone 1 §2 — Edge Detection

"Sobel and Prewitt gradient operators, returning both magnitude and orientation, and a complete Canny detector comprising Gaussian smoothing, gradient computation, non-maximum suppression, double thresholding and hysteresis-based edge linking. Parameter choices must be stated and their effect illustrated on representative pieces."

Requirement	Implementation	Evidence	Status
Sobel, magnitude and orientation	<code>edge_detection.sobel</code>	Verified on a synthetic vertical edge: magnitude peaks at the edge, orientation is normal to it	Met
Prewitt, magnitude and orientation	<code>edge_detection.prewitt</code>	Same verification	Met
Complete Canny, all five stages	<code>edge_detection.canny</code> , with <code>non_maximum_suppression</code> , <code>double_threshold</code> , <code>hysteresis</code> exposed separately	Each stage is individually callable and individually tested	Met
Parameter choices stated	Module and function docstrings	σ , kernel size and the two hysteresis thresholds are documented at their definitions	Met
Effect illustrated on representative pieces	<code>notebooks/full_project_demo.ipynb</code> §4	Sobel and Prewitt magnitudes, the orientation field where the gradient is strong, and all five Canny stages, computed on a real photograph and written to <code>results/edge_visualisations/</code>	Met

The illustration is computed, not stored: running the notebook regenerates every figure from the photograph in `data/input/`. The stage counts it prints make the effect of the parameters explicit — at $\sigma = 1.4$ with thresholds 30 and 70, the 230 396 pixels carrying a non-zero gradient fall to 41 888 after non-maximum suppression and to 11 030 in the final linked edge map.

5. Milestone 1 §3 — Piece Segmentation and Contour Extraction

"Produce a foreground mask separating pieces from the background using the thresholding routines above. Label connected components, from scratch, to assign a distinct identity to each piece. Trace the boundary of each piece. Extract each piece to its bounding box and store its mask, contour, and a normalized orientation."

Requirement	Implementation	Evidence	Status
Foreground mask from the thresholding routines	<code>segmentation.foreground_mask</code>	Run on a real photograph: 26.7% foreground, Otsu threshold 93	Met
Connected components, from scratch	<code>segmentation.connected_components</code>	Run-length + union-find; returns areas, bounding boxes, centroids and per-component masks; tested for 4- versus 8-connectivity and for label merging	Met
Boundary tracing	<code>contour_extraction.trace_boundary</code>	Moore-neighbourhood tracing, matched point-for-point against <code>cv2.findContours</code>	Met
Bounding box, mask, contour, normalised orientation	<code>segmentation.extract_pieces</code> , <code>normalize_orientation</code>	All four available per piece	Met
Supporting morphology	<code>binary_erode/dilate/open/close</code> , <code>fill_holes</code> , <code>propagate_labels</code>	Used to separate pieces that touch in the photograph	Met

Applied across the dataset this yields **1,531 piece observations from 50 photographs**, combined into 35 clean, occlusion-free canonical pieces.

6. Milestone 1 §4 — Piece-Edge Description

"Locate the four corners on the contour and thereby divide the boundary into four sides; classify every side as a tab, a blank, or a flat from its geometry; and sample a strip of colour along the interior of each side to serve as a photometric signature for matching."

Requirement	Implementation	Evidence	Status
Four corners → four sides	<code>piece_geometry.canonicalize, _side_arcs</code>	Pieces are warped so their body corners land on a fixed square; the contour is split into the four arcs between them, named top/right/bottom/left	Met
Tab / blank / flat from geometry	<code>piece_geometry.classify_side</code>	Classified from <i>relief</i> — the mean signed deviation of the side's central half — not peak deviation, which a single nick in the cardboard defeats	Met
Interior colour strip	<code>piece_geometry._sample_colors</code>	Sampled at three depths by default and at any requested depth set; carried inwards where a sample falls outside the silhouette	Met
Runs on the real pieces	—	Verified on all 35 extracted pieces	Met

A measured limitation, reported rather than hidden. Side classification is not perfect on this puzzle. Three of the 35 pieces come out with the wrong number of flat sides, because relief lands in the ambiguous band for pieces 5 and 29 and because piece 2's silhouette is physically damaged by a spring clamp resting on it in all 50 photographs. Five profile statistics and several combinations were swept; the best of them still misclassifies two pieces. §8 describes how the assembly stage was designed to tolerate this rather than depend on it.

7. Milestone 1 §5 — Piece-Edge Matching

"The library must compute a single number that says how good a match a pair of sides is ... The report must state the exact formula you use and how much weight you give to shape versus colour."

The exact formula

For two sides `a` and `b`, with `b` reversed (two mating sides run in opposite physical directions). `x0` is the colour strip sampled just beneath side `x`, `x1` the strip one step deeper:

```

D(a, b) = INCOMPATIBLE                if the sides cannot physically mate
D(a, b) = M(a → b) + M(b → a)        otherwise

M(x → y) = mean_i (r_i - μ)T S-1 (r_i - μ)
g_i = x0_i - x1_i                    x's own colour gradient, from its edge inwards
μ    = mean_i g_i
S    = cov(g) + 1e-4·I                3×3, over the colour channels
r_i  = y0_i - x0_i                    the colour change actually observed across the seam

```

This is Mahalanobis gradient compatibility. It predicts the colour just beyond a side by extrapolating that side's own gradient and penalises the discrepancy under the covariance of that gradient, which makes it invariant to the constant brightness offsets that dominate this puzzle's large white regions.

The weight given to shape versus colour

```

w_colour = 1.0      MGC is the entire numeric score
w_shape  = 0.0      shape is a hard gate only, never a weighted term

```

Shape decides only *whether* two sides may mate — rejecting tab-against-tab, blank-against-blank, and anything involving a flat border edge — and contributes nothing to the number. **This is a measured decision, not a stylistic one.** The puzzle is die-cut: across all 140 sides every tab peaks at $\approx +0.27$ body units and every blank at ≈ -0.25 , so the silhouette says almost nothing about *which* tab belongs in *which* blank. Adding shape to the score makes matching monotonically worse.

Result: rank of the true partner, over the 116 directed true-neighbour pairs

Measure	Rank-1	Top-3	Median rank
Shape only	5.2%	6.0%	28
Colour SSD	22.4%	34.5%	8
Colour SSD, DC removed	19.8%	29.3%	14
Shape 0.6 + colour 0.4	7.8%	19.0%	16
MGC — adopted	46.6%	58.6%	2
MGC 0.95 + shape 0.05	44.0%	56.0%	2
MGC 0.90 + shape 0.10	43.1%	55.2%	2
MGC 0.80 + shape 0.20	34.5%	50.0%	4

Colour space and sampling depth were swept as well: BGR at depths 3/8/14 beat Lab (38.8%), HSV (29.3%), four finer depth sets, two alternative gradient baselines, median-combined residuals, and every blend with DC-removed SSD. Nothing beat 46.6%.

Status: Met. A single number, with the exact formula and the shape-versus- colour weighting stated and justified by measurement.

8. Milestone 1 §6 — Assembly Algorithm

"The compatibility measure is coupled with a greedy best-first search ... The algorithm initialises a placement grid from a corner or border piece and, at each iteration, selects the unused piece and orientation that minimise the total edge dissimilarity with the already-placed neighbours, proceeding until every cell is filled. The report must specify the tie-breaking rule, the handling of dead ends and unplaceable pieces, and must guarantee that the algorithm returns the best arrangement obtained, even when incomplete. Where the difficulty tier includes rotated pieces, each piece's rotation is resolved during placement."

Requirement	How it is fulfilled	Status
Greedy best-first search	A beam search , which is greedy best-first generalised: at width 1 it is greedy best-first. Width 10,000 is the default, for the reason measured in §10	Met
Grid initialised from a corner or border piece	Filling starts at the top-left corner cell. The flat-side cost means only a piece with two adjacent straight edges can sit there cheaply. A fully border-first ring order is also implemented and selectable (<code>order='border'</code>)	Met
Each iteration minimises dissimilarity with placed neighbours	Every candidate is scored as the sum of its seam costs against all already-placed neighbours, plus its flat-side cost	Met
Proceeds until every cell is filled	All 35 cells filled on every run	Met
Tie-breaking rule	Within a cell, candidates are ordered by (cost, piece index, rotation) ascending. Partial arrangements are ordered by cost, equal costs breaking by generation order, which is itself fully determined. The result is independent of the order the pieces are supplied — asserted directly by solving the same puzzle with the piece bag reversed	Met
Dead ends and unplaceable pieces	The tab/blank gate is a bounded penalty, not a filter, so no legal cell is ever empty of candidates. If a cell somehow has none, it is left empty and the search continues	Met
Returns the best arrangement even when incomplete	The returned arrangement is chosen by most-cells-filled first, then lowest cost. Incompleteness is reported in the output as <code>complete: false</code>	Met
Rotation resolved during placement	Every candidate is a (piece, rotation) pair; all four quarter turns of all 35 pieces are searched at every cell	Met

Two design decisions forced by measurement

The flat-side pattern is a cost, not a classification. Hard-classifying each side leaves 13 edge pieces for 16 edge cells (§6), so the border cannot be filled legally and the search dead-ends before it starts. Instead a cell dictates which sides face outwards, and a piece pays for how far its silhouette disagrees. Unambiguous pieces are effectively barred from

the wrong cells; ambiguous ones can be overruled by colour. The counting constraint comes free, because the search places each piece exactly once.

The tab/blank gate is soft. The true arrangement contains one seam the gate rejects — piece 2's damaged silhouette — so with a hard gate the correct answer is unreachable by construction. A rejected pair is charged its raw cost plus ten typical seam costs instead of being removed.

9. The end-to-end routine

"...must provide an end-to-end routine that accepts a scrambled puzzle and returns the reconstructed image together with a numerical measure of reconstruction quality. The testing puzzles may contain pieces presented in different orientations, so the reconstruction algorithm must correctly determine and resolve the rotation of each piece."

```
python3 main.py --pieces data/pieces          # general entry point
python3 -m scripts.solve_blind                 # shuffles + randomly rotates first
```

The solver receives an unordered bag of pieces with no identity, position or orientation information, and is not told the grid shape. It returns the reconstructed image, every piece's cell and rotation, and a quality score.

The quality score requires no ground truth. It is the fraction of realised seams that are *mutually best* pairs — both sides preferring each other over every alternative in the puzzle. Best-buddy agreement is the standard blind proxy for reconstruction quality, and it is informative here: of the 21 mutually-best pairs the measure finds, 17 are true neighbours (81% precision).

Status: Met. Scrambled puzzle in; image, placements and quality score out; rotation searched and resolved.

10. Results

Reconstruction accuracy

Blind, at the default settings, ≈ 32 s, scored against the derived ground truth only after the search has finished:

Measure	Result
Grid shape inferred	5×7 — correct, and not supplied
Every cell filled	Yes
Neighbour accuracy	34 / 58 seams — 58.6%
Position accuracy	24 / 35 cells — 68.6%
Orientation accuracy	23 / 35 cells — 65.7%
Blind quality score	0.172
Complete reconstruction	No

Accuracy is quoted after a 180° rotation of the whole grid, the one symmetry a non-square rectangle's seams cannot distinguish. Both readings are printed by the solver, so the choice is visible rather than silently favourable.

What each design change was worth

Stage	Neighbours	Positions
First working version — hard flat gate, border-first order, beam 200	19/58 (32.8%)	4/35 (11.4%)
Flat pattern as a cost	21/58 (36.2%)	10/35 (28.6%)
Tuned flat weight and cell order	26/58 (44.8%)	16/35 (45.7%)
Soft gate and beam 10,000	34/58 (58.6%)	24/35 (68.6%)

What limits the result — measured, not assumed

The true arrangement was scored under the solver's own objective and compared with what the search returns:

Beam width	Objective reached	Mean seam cost	Position accuracy	Runtime
3,000	1735	23.83	62.9%	7.5 s
10,000 — default	1713	23.40	68.6%	29 s
30,000	1688	23.22	54.3%	112 s
60,000	1671	22.03	54.3%	245 s
<i>the true arrangement</i>	<i>1943</i>	<i>19.47</i>	<i>100%</i>	—

Two conclusions follow, and both matter for interpreting the accuracy above.

The search works. The objective it reaches falls monotonically as the beam widens, and from width 3,000 upwards it finds arrangements that score *better* than the correct one.

The compatibility measure is the limitation. Precisely because the truth is not the optimum of the objective, minimising the objective harder does not make the answer more correct — accuracy wanders between 54% and 69% instead of tracking the objective down. No amount of extra search closes this gap.

On a synthetic puzzle cut from a known image, where the measure is not the bottleneck, the same search reconstructs the puzzle **exactly**. That is asserted by `tests/test_assembly.py`, and it is what makes the attribution above sound rather than speculative.

Ground truth, and how it was verified

The layout used for scoring was derived from the photographs and then tested, not assumed. On shape evidence alone — flat borders and tab/blank complementarity, no colour — the derived layout violates **5 of 82** constraints against a minimum of **34** across 300 random arrangements, and no random arrangement came close. `results/ground_truth/solved_reference.png` confirms it visually: the HIWIN logo and the red 飛躍新 30+ mark read correctly and the artwork is continuous across seams.

11. How the last conformance checks were closed

Three gaps stood open when this report was first written; all three are now met.

Gap	Closed by
\$2 — effect of edge parameters not illustrated	<code>notebooks/full_project_demo.ipynb</code> \$4 computes the Sobel/Prewitt magnitudes, the orientation field and all five Canny stages from a real photograph and writes them to <code>results/edge_visualisations/</code>
Four of the seven prescribed test filenames absent	<code>tests/test_enhancement.py</code> (20 tests), <code>tests/test_thresholding.py</code> (11), <code>tests/test_edge_detection.py</code> (16) and <code>tests/test_piece_description.py</code> (17) were written. They are not renamed stubs: the convolution is checked against an explicit per-pixel double loop, the median filter against a plain neighbourhood median and across its internal banding, adaptive thresholding on an illumination ramp that defeats every global threshold, and the description on all 35 real pieces
Six prescribed folders absent	<code>data/input/</code> , <code>data/sample_pieces/</code> , <code>results/enhanced_images/</code> , <code>results/masks/</code> , <code>results/contours/</code> and <code>results/edge_visualisations/</code> now exist and are populated — the first two with the photograph and the sample pieces the notebook demonstrates on, the rest by running it
Notebook crashed on cell 3	Rewritten. It now runs top to bottom in ≈ 70 s and reproduces the 46.6% matcher figure and the 24/35 reconstruction figure quoted above

The test suite grew from 55 to 108 in the process, and the superseded stack (`src/pipeline.py`, `src/dataset_reconstruction.py`, `src/solver_v2.py`, `src/assembly.py`, `src/piece_description.py`, `src/edge_matching.py`, `src/rendering.py`) was deleted along with the four contradictory status documents it was described in. `STATUS.md` replaces them.

12. Milestone 2 status — substantially outstanding

Reported in full in [milestone_2_report.md](#); the short version, because it bears on §10's conclusion that a better pairwise measure is what this puzzle needs.

Requirement	Status
Dataset preparation — positive and negative side-pair samples	Done. <code>ml/dataset.py</code> : 84 training positives, 672 negatives, 20 validation positives, held out as a contiguous 2×4 block of pieces so no piece appears on both sides of the split
Siamese CNN — implemented	Done. <code>ml/siamese.py</code>
Graph Neural Network — implemented	Done. <code>ml/gnn.py</code>
Siamese CNN — trained	Partly. 500 epochs; only the <code>neighbor_logit</code> head. <code>orientation_logits</code> and <code>compatibility</code> remain at random initialisation and must not be reported
GNN — trained	Partly. 500 epochs on the same split, same restriction to one head
Same dataset split for both models	Done. The GNN's edges are built from the Siamese's own examples, and <code>tests/test_ml.py</code> asserts it
Model scores passed to the assembly algorithm	Not done
Performance comparison against the classical method	Done for the pairwise ranking only. <code>python3 -m ml.evaluate_models</code>

What the training established

Rank-1 over the held-out block, every measure scored on identical gated candidate sets by `ml/ranking.py`:

Measure	Rank-1	Top-3	Median rank
Siamese CNN, epoch 500	40.0%	70.0%	2
GNN, epoch 500	40.0%	70.0%	2
Classical MGC	55.0%	80.0%	1
GNN, epoch 225 (best validation loss)	60.0%	80.0%	1

Both models overfit hard — best validation loss around epoch 223–225, then an order-of-magnitude degradation while training loss collapses towards zero (the GNN reaches 0.0006 by epoch 500, which is memorisation of 756 edges). At the checkpoints the runs actually keep, **neither beats the classical measure.** The GNN's best-validation checkpoint does, at 60.0% against 55.0% — but that is 12 correct pairs against 11 out of 20, well inside noise, and not the checkpoint the run selects.

Those rank figures are over the held-out block only and are **not** comparable to the 46.6% quoted in §7, which ranks against all 140 sides; the classical baseline is recomputed on the identical restricted candidate set so the rows can be compared with each other.

With one puzzle and 84 training positives, this is the expected outcome. Scaling the training set — `src/synthetic.py` cuts puzzles of any size from any image, with the true adjacencies known by construction — and selecting the best-validation epoch rather than the last are the two prerequisites for a learned measure that could beat MGC. Beating MGC is the thing that would raise the 68.6% in §10.

13. Reproducing every figure in this report

```
python3 -m pytest -q                # 108 tests, ~15 s
python3 -m scripts.audit_requirements # 24/24 conformance
python3 -m scripts.build_ground_truth # rebuild ground truth from the dataset, ~170
s
python3 -m scripts.solve_blind       # blind reconstruction, ~32 s
python3 -m scripts.evaluate_solver   # the §10 limiting-factor tables, ~6 min
python3 -m scripts.experiment_mgc    # the §7 measure table, ~90 s
python3 -m scripts.experiment_matcher # the §7 colour/depth sweep, ~10 min
python3 -m ml.train_siamese --epochs 500 # the §12 Siamese run, ~140 s
python3 -m ml.train_gnn --epochs 500    # the §12 GNN run, ~94 s
python3 -m ml.evaluate_models         # the §12 comparison, instant
```

Implementation: `src/enhancement.py`, `src/thresholding.py`, `src/edge_detection.py`, `src/segmentation.py`, `src/contour_extraction.py`, `src/piece_geometry.py`, `src/edge_compatibility.py`, `src/solver.py`, `src/evaluation.py`, `ml/dataset.py`, `ml/train_siamese.py`.